

1. PRÉSENTATION DU PROJET

1.1 Contexte

Ce projet consiste à développer une application Java pour simuler la partie radio d'un réseau GSM (Global System for Mobile Communications). Le réseau GSM est décomposé en cellules, chaque cellule étant contrôlée par une station de base (BTS - Base Transceiver Station). Les utilisateurs mobiles (MS - Mobile Station) s'attachent à une BTS pour communiquer.

La communication se fait via deux liaisons :

Liaison montante (Uplink) : du mobile vers la BTS

Liaison descendante (Downlink) : de la BTS vers le mobile

1.2 Objectifs

- Modéliser les entités d'un réseau GSM en Java
- Appliquer les concepts de Programmation Orientée Objet (POO)
- Gérer l'attachement des utilisateurs aux BTS
- Simuler des appels entre utilisateurs
- Détecter et gérer les erreurs (cellule saturée, utilisateur introuvable)
- Organiser le code dans un package unique reseau

2. ARCHITECTURE DU PROJET

2.1 Structure du projet

Le projet est organisé en un seul package reseau contenant 9 fichiers :

ProjetGSM/

└─ src/

└─ reseau/

└─ InterfaceMS.java

└─ EtatCelluleException.java

└─ MSIntrouvableException.java

└─ BTS.java

└─ MS.java

```

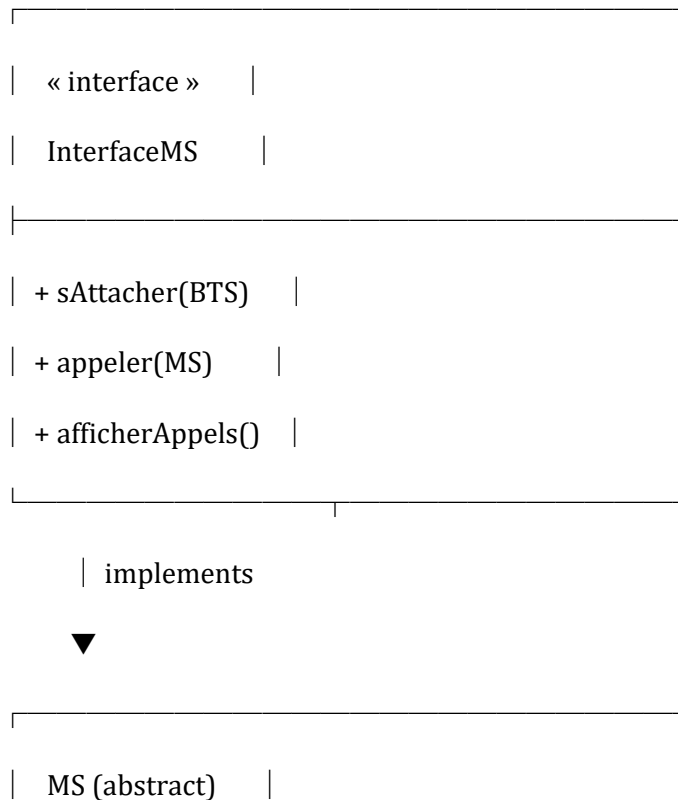
└─ Smartphone.java
└─ Tablette.java
└─ Reseau.java
└─ TestReseau.java

```

2.2 Rôle de chaque classe

- ✓ InterfaceMS.java : Interface : Contrat définissant les méthodes obligatoires pour toute station mobile
- ✓ EtatCelluleException.java : Exception : Levée quand une cellule BTS est saturée
- ✓ MSIntrouvableException.java : Exception : Levée quand un utilisateur ou une BTS est introuvable
- ✓ MS.java Classe abstraite : Classe mère de tous les équipements mobiles
- ✓ Smartphone.java : Classe concrète Hérite de MS, représente un smartphone
- ✓ Tablette.java : Classe concrète Hérite de MS, représente une tablette
- ✓ BTS.java : Classe concrète Station de base gérant une cellule
- ✓ Reseau.java : Classe concrète Réseau GSM complet contenant les BTS
- ✓ TestReseau.java : Programme principal Point d'entrée main(), exécute tous les tests

2.3 Diagramme de classes simplifié



|_____|

| # nom, prenom |

| # msisdn, imsi |

| # btsAttache |

| # appelsRecus |

|_____|

| + sAttacher(BTS) |

| + appeler(MS) |

| + afficherType() abs |

|_____|

| |

▼ ▼

|_____| |_____|

| Smartphone | | Tablette |

|_____| |_____|

| - os | | - sim |

| - ecran | | - ecran |

|_____| |_____|

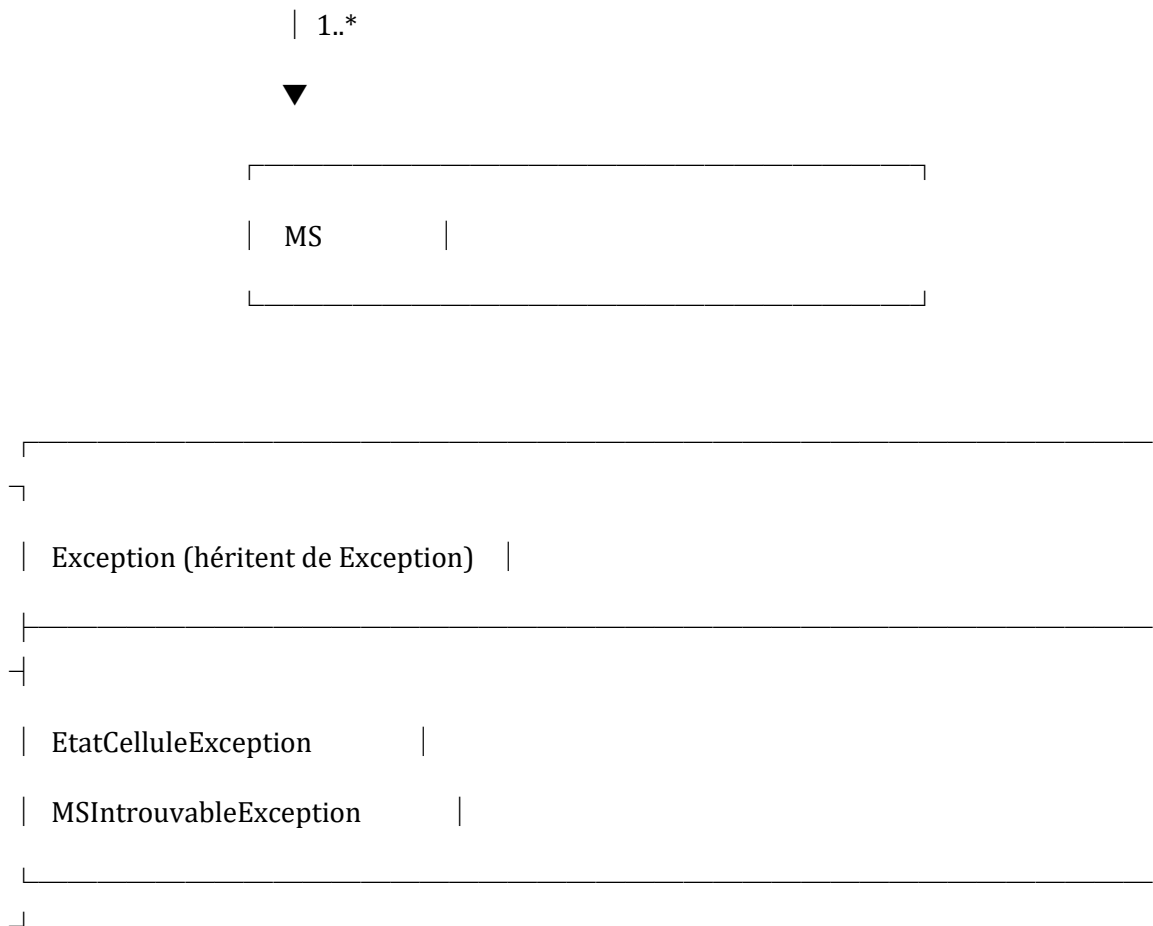
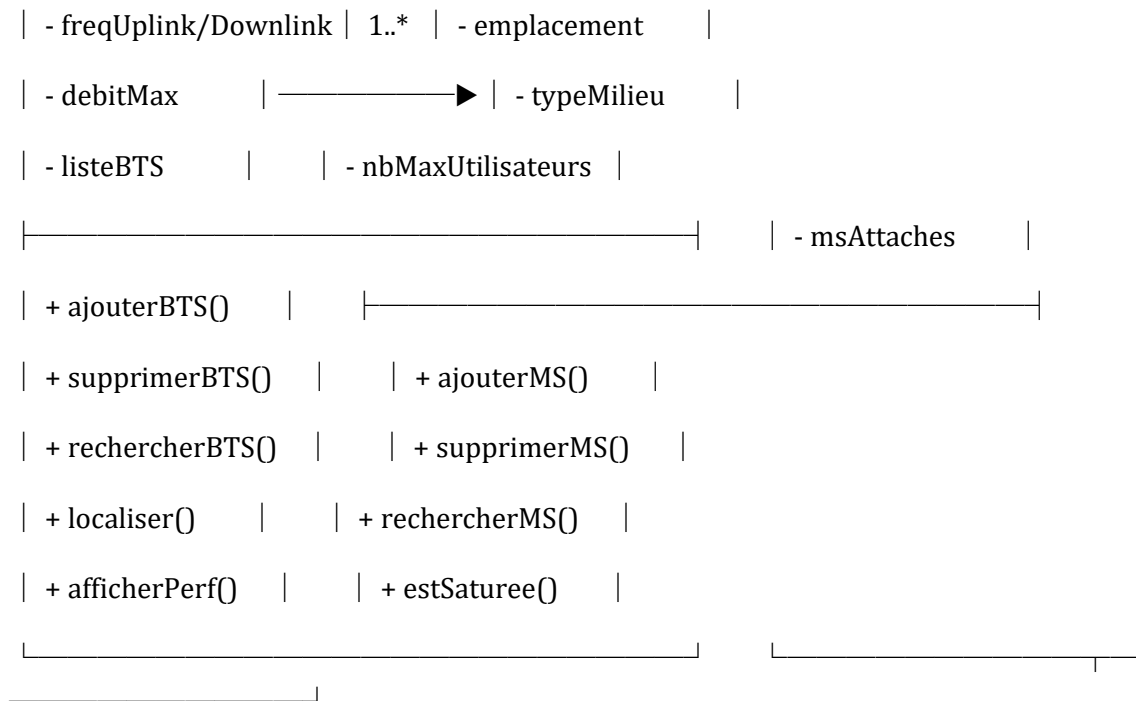
|_____|

| Reseau | | BTS |

|_____|

|_____|

| - nom | | - numero |



3. NOTIONS POO UTILISÉES

Notion Application dans le projet

- ✓ Encapsulation : Attributs privés/protégés avec getters publics. Ex : private String numero dans BTS, accessible via getNumero()
- ✓ Héritage Smartphone et Tablette héritent de MS via le mot-clé extends
- ✓ Classe abstraite : MS déclarée abstract avec la méthode abstraite afficherType() que les sous-classes doivent implémenter
- ✓ Interface : InterfaceMS définit le contrat avec 3 méthodes : sAttacher(), appeler(), afficherAppelsRecus()
- ✓ Polymorphisme : Une variable de type MS peut référencer un Smartphone ou une Tablette. L'appel à afficherType() exécute la version correspondante
- ✓ Collections : ArrayList<MS> pour stocker les utilisateurs attachés à une BTS, ArrayList<BTS> pour les BTS du réseau
- ✓ Exceptions : 2 exceptions personnalisées : EtatCelluleException (saturation) et MSIntrouvableException (recherche infructueuse)
- ✓ Redéfinition (Override) : toString(), afficherType(), sAttacher(), appeler() redéfinies dans les sous-classes

4. DÉTAIL DES CLASSES

4.1 Interface InterfaceMS

L'interface InterfaceMS définit le contrat que toute station mobile doit respecter. Elle impose 3 méthodes :

java

```
public interface InterfaceMS {  
  
    boolean sAttacher(BTS bts);    // Attacher le mobile à une BTS  
  
    boolean appeler(MS destinataire); // Appeler un autre mobile  
  
    void afficherAppelsRecus();    // Afficher l'historique des appels  
  
}
```

4.2 Classe MS (Mobile Station) - Classe Abstraite

La classe MS est le cœur du modèle. Elle représente un utilisateur mobile quelconque.

Attributs :

Attribut	Type	Visibilité	Description
nom	String	protected	Nom de l'utilisateur
prenom	String	protected	Prénom de l'utilisateur
motDePasse	String	protected	Mot de passe
msisdn	String	protected	Numéro de téléphone (MSISDN)
imsi	String	protected	Identifiant unique de la carte SIM (IMSI)
btsAttache	BTS	protected	BTS à laquelle le mobile est attaché (null si aucune)
appelsRecus	ArrayList<String>	protected	Liste des identifiants des appelants

Méthodes principales :

Méthode	Description
sAttacher(BTS bts)	Attache le mobile à une BTS. Vérifie que la cellule n'est pas saturée. Gère le détachement automatique de l'ancienne BTS
appeler(MS destinataire)	Effectue un appel. Vérifie que l'appelant et le destinataire sont attachés à une BTS
recevoirAppel(String)	Enregistre un appel reçu dans le journal
afficherAppelsRecus()	Affiche l'historique complet des appels reçus
afficherCaracteristiques()	Affiche la fiche complète de l'utilisateur
afficherType()	Méthode abstraite : chaque sous-classe doit la redéfinir

Sous-classes (Héritage) :

Smartphone : ajoute systemeExploitation et tailleEcran

Tablette : ajoute supportCarteSim et tailleEcran

4.3 Classe Smartphone

Hérite de MS. Représente un téléphone intelligent.

Attributs supplémentaires :

Attribut	Type	Description
systemeExploitation	String	Ex: "Android 14", "iOS 17"
tailleEcran	double	Taille en pouces (ex: 6.5)

Redéfinition de afficherType() :

```
java
@Override
public void afficherType() {
    System.out.println("Smartphone | OS : " + systemeExploitation +
        " | Ecran : " + tailleEcran + "\"");
}
```

4.4 Classe Tablette

Hérite de MS. Représente une tablette.

Attributs supplémentaires :

Attribut	Type	Description
supportCarteSim	boolean	true si la tablette accepte une carte SIM
tailleEcran	double	Taille en pouces (ex: 10.9)

Redéfinition de afficherType() :

```
java
```

```

@Override

public void afficherType() {

    System.out.println("Tablette | SIM : " + (supportCarteSim ? "Oui" : "Non") +
        " | Ecran : " + tailleEcran + "\"");
}

```

4.5 Classe BTS (Base Transceiver Station)

La classe BTS représente une station de base qui couvre une cellule géographique.

Attributs :

Attribut	Type	Description
numero	String	Identifiant unique (ex: "BTS-001")
emplacement	String	Localisation géographique
hauteur	double	Hauteur de l'antenne en mètres
typeMilieu	String	"urbain" ou "rural"
rayonCouverture	double	Rayon de la cellule en km
puissanceEmission	double	Puissance en dBm
nbMaxUtilisateurs	int	Capacité maximale d'utilisateurs
msAttaches	ArrayList<MS>	Liste des utilisateurs attachés

Méthodes principales :

Méthode	Description
ajouterMS(MS)	Ajoute un utilisateur si la capacité n'est pas atteinte
supprimerMS(MS)	Retire un utilisateur de la liste
rechercherMS(String)	Recherche par MSISDN. Lève MSIntrouvableException si absent
estSaturee()	Retourne true si le nombre max d'utilisateurs est atteint

afficherInfos() Affiche l'état complet de la BTS et la liste des utilisateurs

4.6 Classe Reseau

La classe Reseau représente le réseau GSM complet d'un opérateur.

Attributs :

Attribut	Type	Description
nom	String	Nom de l'opérateur (ex: "Orange")
frequenceUplink	double	Fréquence montante en MHz
frequenceDownlink	double	Fréquence descendante en MHz
typeAccesMultiple	String	FDMA, TDMA, CDMA, OFDMA
debitMaxUplink	double	Débit max montant en Mbps
debitMaxDownlink	double	Débit max descendant en Mbps
delaiMax	double	Délai maximum en ms
listeBTS	ArrayList<BTS>	Liste des BTS du réseau

Méthodes principales :

Méthode	Description
ajouterBTS(BTS)	Ajoute une BTS au réseau
supprimerBTS(String)	Supprime une BTS par son numéro
rechercherBTS(String)	Recherche une BTS. Lève MSIntrouvableException si absente
calculerNombreBTSParMilieu(String)	Compte les BTS par type de milieu ("urbain"/"rural")
calculerNombreAbonnesInscrits()	Compte le total d'utilisateurs sur toutes les BTS
localiserUtilisateur(String)	Trouve la BTS où se trouve un utilisateur par son MSISDN
afficherPerformances()	Affiche le tableau de bord complet du réseau

4.7 Exceptions personnalisées

EtatCelluleException :

java

```
public class EtatCelluleException extends Exception {  
    public EtatCelluleException(String message) {  
        super(message);  
    }  
}
```

Levée quand on tente d'attacher un utilisateur à une BTS déjà saturée.

MSIntrouvableException :

java

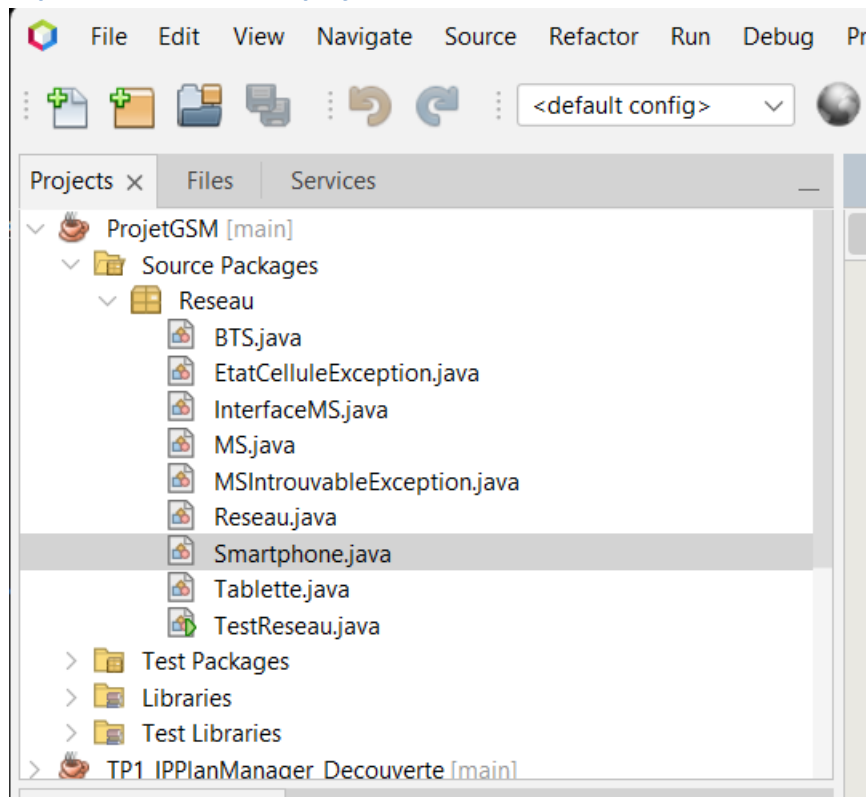
```
public class MSIntrouvableException extends Exception {  
    public MSIntrouvableException(String message) {  
        super(message);  
    }  
}
```

Levée quand une recherche d'utilisateur ou de BTS échoue.

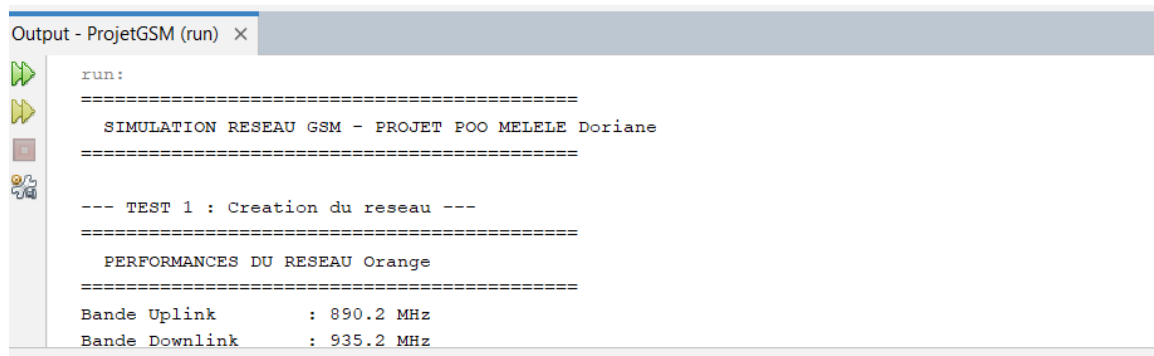
6. CAPTURES D'ÉCRAN

Instructions : Exécute TestReseau.java et prends les captures aux moments indiqués.

Capture 1 : Structure du projet dans l'IDE



Capture 2 : En-tête du programme



Capture 3 : Création du réseau et ajout des BTS (Tests 1-2)

```
Output - ProjetGSM (run) ×
=====
--- TEST 1 : Creation du reseau ---
=====
PERFORMANCES DU RESEAU Orange
=====
Bande Uplink      : 890.2 MHz
Bande Downlink    : 935.2 MHz
Acces multiple    : OFDMA
Debit max Uplink  : 150.0 Mbps
Debit max Downlink : 300.0 Mbps
Delai max         : 10.0 ms
-----
Nombre total de BTS : 0
BTS en zone urbaine : 0
BTS en zone rurale  : 0
Abonnes inscrits   : 0
=====

--- TEST 2 : Creation des BTS ---
RESEAU Orange : BTS BTS-001 ajoutee.
RESEAU Orange : BTS BTS-002 ajoutee.
RESEAU Orange : BTS BTS-003 ajoutee.

--- TEST 3 : Creation des utilisateurs ---
=====
CARACTERISTIQUES DE L'UTILISATEUR
-----
Nom      : Diop
Prenom   : Mamadou
MSISDN (Tel) : 771234567
IMSI (SIM) : IMSI001
BTS attachee : Aucune
Type     : Smartphone | OS : Android | Ecran : 6.5 pouces
=====
```

Capture 4 : Attachement des utilisateurs et saturation (Test 4)

```
=====

--- TEST 4 : Attachement aux BTS ---
SUCCES : 771234567 maintenant attache a la BTS BTS-001
SUCCES : 778765432 maintenant attache a la BTS BTS-001
SUCCES : 770000001 maintenant attache a la BTS BTS-001
ECHEC attachement : Cellule saturee : impossible d'attacher 770000002 a la BTS BTS-001
SUCCES : 770000002 maintenant attache a la BTS BTS-002
```

Capture 5 : Appels entre utilisateurs (Tests 5-6)

```
--- TEST 5 : Recherche utilisateur ---
Trouve : Diop Mamadou

--- TEST 6 : Appels ---
APPEL EN COURS : 771234567 -> 778765432
APPEL EN COURS : 778765432 -> 770000001
APPEL EN COURS : 770000002 -> 771234567
```

Capture 6 : Journal des appels reçus (Test 7)

```
--- TEST 7 : Journal appels recus ---
-----
Journal d'appels recus pour 771234567 (Diop Mamadou) :
  1. 770000002 (Sarr Fatou)
-----

-----
Journal d'appels recus pour 778765432 (Ndiaye Awa) :
  1. 771234567 (Diop Mamadou)
-----

-----
Journal d'appels recus pour 770000001 (Fall Ibrahima) :
  1. 778765432 (Ndiaye Awa)
-----

-----
Journal d'appels recus pour 770000002 (Sarr Fatou) :
  Aucun appel reçu.
-----
```

Capture 7 : État des BTS avec saturation visible (Test 9)

```
--- TEST 9 : Performances ---
=====
PERFORMANCES DU RESEAU Orange
=====
Bande Uplink      : 890.2 MHz
Bande Downlink    : 935.2 MHz
Acces multiple    : OFDMA
Debit max Uplink  : 150.0 Mbps
Debit max Downlink : 300.0 Mbps
Delai max         : 10.0 ms
-----

Nombre total de BTS : 3
BTS en zone urbaine : 2
BTS en zone rurale  : 1
Abonnes inscrits   : 4
=====
```

Capture 8 : Performances finales du réseau (Test 10)

```
--- TEST 10 : Etat des BTS ---
=====
BTS Numero      : BTS-001
Emplacement     : Dakar Centre
Type de milieu  : urbain
Hauteur        : 35.0 m
Rayon couverture: 2.0 km
Puissance emis. : 46.0 dBm
Utilisateurs    : 3 / 3
Etat cellule    : SATURÉ
-----

Liste des utilisateurs attaches :
- Diop Mamadou | 771234567
- Ndiaye Awa | 778765432
- Fall Ibrahima | 770000001
=====

=====
BTS Numero      : BTS-002
Emplacement     : Thies
Type de milieu  : rural
Hauteur        : 55.0 m
Rayon couverture: 8.0 km
Puissance emis. : 43.0 dBm
Utilisateurs    : 1 / 2
Etat cellule    : DISPONIBLE
-----

Liste des utilisateurs attaches :
- Sarr Fatou | 770000002
=====

=====
BTS Numero      : BTS-003
Emplacement     : Pikine
Type de milieu  : urbain
Hauteur        : 28.0 m
```

Capture 9 : Gestion des exceptions et fin du programme (Test 11)

```
=====

--- TEST 11 : Exception ---
Exception : BTS BTS-999 introuvable dans le reseau Orange

=====

FIN DE LA SIMULATION MELELE Doriane
=====

BUILD SUCCESSFUL (total time: 0 seconds)
```

7. CONCLUSION

Ce projet a permis de mettre en pratique les concepts fondamentaux de la Programmation Orientée Objet dans un contexte concret de télécommunications.

La modélisation d'un réseau GSM illustre comment les objets Java peuvent représenter des entités réseau complexes et leurs interactions.

L'organisation en un package unique `reseau` contenant 9 classes permet de centraliser le code tout en maintenant une séparation claire des responsabilités.

Les notions suivantes ont été appliquées avec succès :

Notion Application dans le projet

Interface `InterfaceMS` définit le contrat que toute station mobile doit respecter

Classe abstraite `MS` avec la méthode abstraite `afficherType()`

Héritage `Smartphone` et `Tablette` héritent de `MS`

Polymorphisme Une variable `MS` peut référencer un `Smartphone` ou une `Tablette`

Encapsulation Attributs privés avec getters publics

Collections `ArrayList` pour gérer les listes de BTS et d'utilisateurs

Exceptions `EtatCelluleException` et `MSIntrouvableException`

Le projet est entièrement fonctionnel, documenté et disponible sur GitHub à l'adresse :